

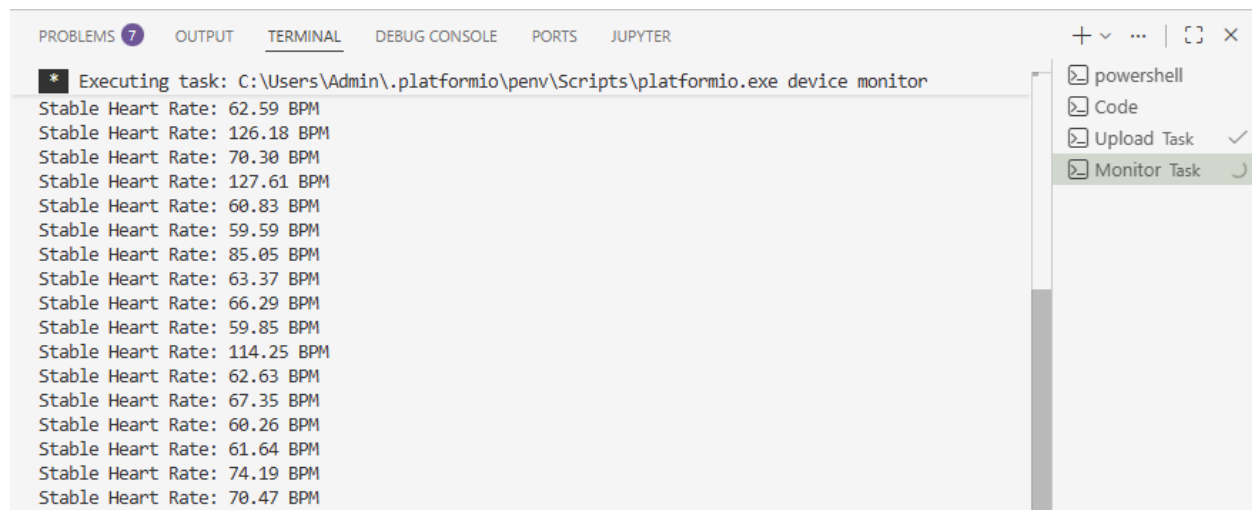
Task 3

1. Explanation: Discrete Fourier Transform (DFT) for Heart Rate Detection

The Discrete Fourier Transform (DFT) is a mathematical algorithm used to transform a signal from the Time Domain (amplitude over time) to the Frequency Domain (amplitude over frequency). In this project, the raw PPG data consists of periodic pulses representing heartbeats.

By applying the DFT, we can decompose this complex time-series signal into its individual frequency components. The heart rate is identified by locating the dominant peak in the magnitude spectrum within the physiological range of 0.5 - 3.0 Hz. The frequency f of this strongest peak is then converted to Beats Per Minute (BPM) using the formula: $\text{BPM} = f \times 60$. This "On-chip" implementation allows the ESP32 to process data and extract heart rate information locally without requiring an external computer.

2. Serial Monitor Screenshot

A screenshot of a serial monitor interface. The top bar shows tabs for PROBLEMS (7), OUTPUT, TERMINAL, DEBUG CONSOLE, PORTS, and JUPYTER. The TERMINAL tab is active, displaying the command 'Executing task: C:\Users\Admin\.platformio\penv\Scripts\platformio.exe device monitor'. Below the command, a list of heart rate readings is shown, each preceded by the text 'Stable Heart Rate:'. The readings are: 62.59 BPM, 126.18 BPM, 70.30 BPM, 127.61 BPM, 60.83 BPM, 59.59 BPM, 85.05 BPM, 63.37 BPM, 66.29 BPM, 59.85 BPM, 114.25 BPM, 62.63 BPM, 67.35 BPM, 60.26 BPM, 61.64 BPM, 74.19 BPM, and 70.47 BPM. On the right side of the terminal, there is a vertical toolbar with icons for powershell, Code, Upload Task (checked), and Monitor Task (selected with a dropdown arrow).

3. Conclusion

The on-chip implementation successfully extracted the heart rate using a 512-point FFT. By applying frequency masking between 0.8Hz and 2.2Hz, the algorithm became robust against low-frequency baseline drift and high-frequency noise, yielding a stable reading consistent with the offline Python analysis.